

Deep copy & shallow copy



Shallow Copy

Explanation : A shallow copy creates a new object, but only copies the first level of properties. If the original object contains nested objects or arrays, the copy will reference the same nested objects in memory.

```
// Example 1: Shallow copy with spread operator
const original = {
  name: "John",
  age: 30,
  address: {
    city: "Mumbai",
    country: "India"
  }
};

const shallowCopy = { ...original };

// Modifying first-level property
shallowCopy.name = "Alice";
console.log(original.name); // "John" (not affected)

// Modifying nested property
shallowCopy.address.city = "Delhi";
console.log(original.address.city); // "Delhi" (affected!)
```

Other Shallow Copy Methods:

Explanation : Why does this happen? The address object is only copied by reference, so both original and shallowCopy point to the same address object in memory.

```
// Using Object.assign()
const copy1 = Object.assign({}, original);

// Using spread operator
const copy2 = { ...original };

// For arrays
const arr = [1, 2, [3, 4]];
const arrCopy = [...arr];
arrCopy[2][0] = 99;
console.log(arr[2][0]); // 99 (nested array affected)
```

Deep Copy

Explanation : A deep copy creates a completely independent clone, copying all levels including nested objects and arrays. Changes to the copy won't affect the original.

```
// Method 1: Using structuredClone() (modern, recommended)
const original = {
  name: "John",
  age: 30,
  address: {
    city: "Mumbai",
    country: "India"
  },
  hobbies: ["reading", "coding"]
};

const deepCopy = structuredClone(original);

deepCopy.address.city = "Delhi";
deepCopy.hobbies.push("gaming");

console.log(original.address.city); // "Mumbai"
console.log(original.hobbies); // ["reading", "coding"]
```

Other Deep Copy Methods:

Explanation : A deep copy creates a completely independent clone, copying all levels including nested objects and arrays. Changes to the copy won't affect the original.

```
// Method 2: JSON.parse + JSON.stringify (simple but has limitations)
const deepCopy2 = JSON.parse(JSON.stringify(original));

// Limitations of JSON method:
const problematic = {
  date: new Date(),
  func: () => console.log("hello"),
  undef: undefined,
  symbol: Symbol("id")
};

const jsonCopy = JSON.parse(JSON.stringify(problematic));
console.log(jsonCopy);
// { date: "2026-01-17T..." } - functions, undefined, symbols are lost!
```

Other Deep Copy Methods:

```
// Method 3: Custom recursive function (for complete control)
function deepClone(obj) {
  if (obj === null || typeof obj !== "object") {
    return obj;
  }

  if (obj instanceof Date) {
    return new Date(obj.getTime());
  }

  if (obj instanceof Array) {
    return obj.map(item => deepClone(item));
  }

  const clonedObj = {};
  for (let key in obj) {
    if (obj.hasOwnProperty(key)) {
      clonedObj[key] = deepClone(obj[key]);
    }
  }
  return clonedObj;
}
```

Quick Comparison

```
const data = {  
  level1: "first",  
  nested: {  
    level2: "second"  
  }  
};  
  
// Shallow copy  
const shallow = { ...data };  
shallow.nested.level2 = "CHANGED";  
console.log(data.nested.level2); // "CHANGED"  
  
// Deep copy  
const deep = structuredClone(data);  
deep.nested.level2 = "CHANGED";  
console.log(data.nested.level2); // "second"
```

Quick Comparison

```
const data = {  
  level1: "first",  
  nested: {  
    level2: "second"  
  }  
};  
  
// Shallow copy  
const shallow = { ...data };  
shallow.nested.level2 = "CHANGED";  
console.log(data.nested.level2); // "CHANGED"  
  
// Deep copy  
const deep = structuredClone(data);  
deep.nested.level2 = "CHANGED";  
console.log(data.nested.level2); // "second"
```

Interview Answer

Shallow Copy: Creates a new object and copies only the top-level properties. Nested objects/arrays are copied by reference, meaning both original and copy point to the same nested objects in memory.

Deep Copy: Creates a completely independent clone of an object, recursively copying all nested objects and arrays. Changes to the copy don't affect the original.

The main difference lies in how they handle nested objects:

- **Shallow Copy:** Copies references to nested objects (shared memory)
- **Deep Copy:** Creates new copies of nested objects (separate memory)

Shallow Copy Methods

1. Spread Operator `{...obj}` or `[...arr]`
2. `Object.assign()`
`Object.assign({}, obj)`
3. `Array.slice()` `arr.slice()`
4. `Array.concat()` `arr.concat([])`

Deep Copy Methods

1. `structuredClone()` - Modern, recommended (ES2021)
2. `JSON.parse(JSON.stringify())` - Simple but has limitations
3. `Lodash _.cloneDeep()` - Third-party library
4. Custom recursive function - Full control